INGENIERÍA DEL SOFTWARE II

- Tema 1. Requisitos (Parte 2) -

Grado en Ingeniería Informática - Curso 2026/2027

Rocío Carratalá Sáez (rocio.carratala@uv.es)

Contenido

1. Introducción
2. Métodos de captación de requisitos
3. Modelos de representación
4. Requisitos de usuario y de software
5. Requisitos funcionales y no funcionales
6. Requisitos de capacidad y restricciones de uso
7. Documentación y revisión de requisitos
8. Indicadores clave y resultados
9. Perspectivas de trabajo, personas y acuerdos
10. Historias de usuario
11. Product backlog y priorización

1.3 Modelos de representación

Un modelo es una **simplificación** construida para responder preguntas.

Un modelo no se elige porque “toca hacer un diagrama”, sino porque **permite razonar, comunicar o verificar algo mejor que una lista de frases**.

1.3 Modelos de representación

Un modelo es una **simplificación** construida para responder preguntas.

Un modelo no se elige porque “toca hacer un diagrama”, sino porque **permite razonar, comunicar o verificar algo mejor que una lista de frases**.

- PREDICTIVO: la cadena suele **consolidarse y revisarse** como conjunto **antes** de establecer una línea base. Los requisitos de nivel superior justifican los de software y componente.
- ADAPTATIVO: el objetivo y los resultados esperados **orientan el producto**, mientras requisitos de usuario y software se detallan progresivamente alrededor de incrementos; la trazabilidad puede apoyarse en enlaces entre objetivos, elementos de backlog, decisiones y pruebas.

1.3 Modelos de representación

Pregunta	Modelo posible	Hallazgo buscado
¿Dónde acaba CampusLab?	Diagrama de contexto	Actores, sistemas externos y responsabilidades.
¿Cómo se realiza una tarea?	Flujo de actividad o recorrido	Pasos, decisiones, esperas y excepciones.
¿Qué significan los conceptos?	Modelo de dominio y glosario	Ambigüedades, relaciones y reglas.
¿Cómo cambia una reserva?	Máquina de estados	Transiciones imposibles o eventos omitidos.
¿Qué valor se entrega primero?	Mapa de historias	Flujo completo y cortes incrementales.
¿Se entiende la interacción?	Boceto o prototipo	Errores de comprensión antes de construir.

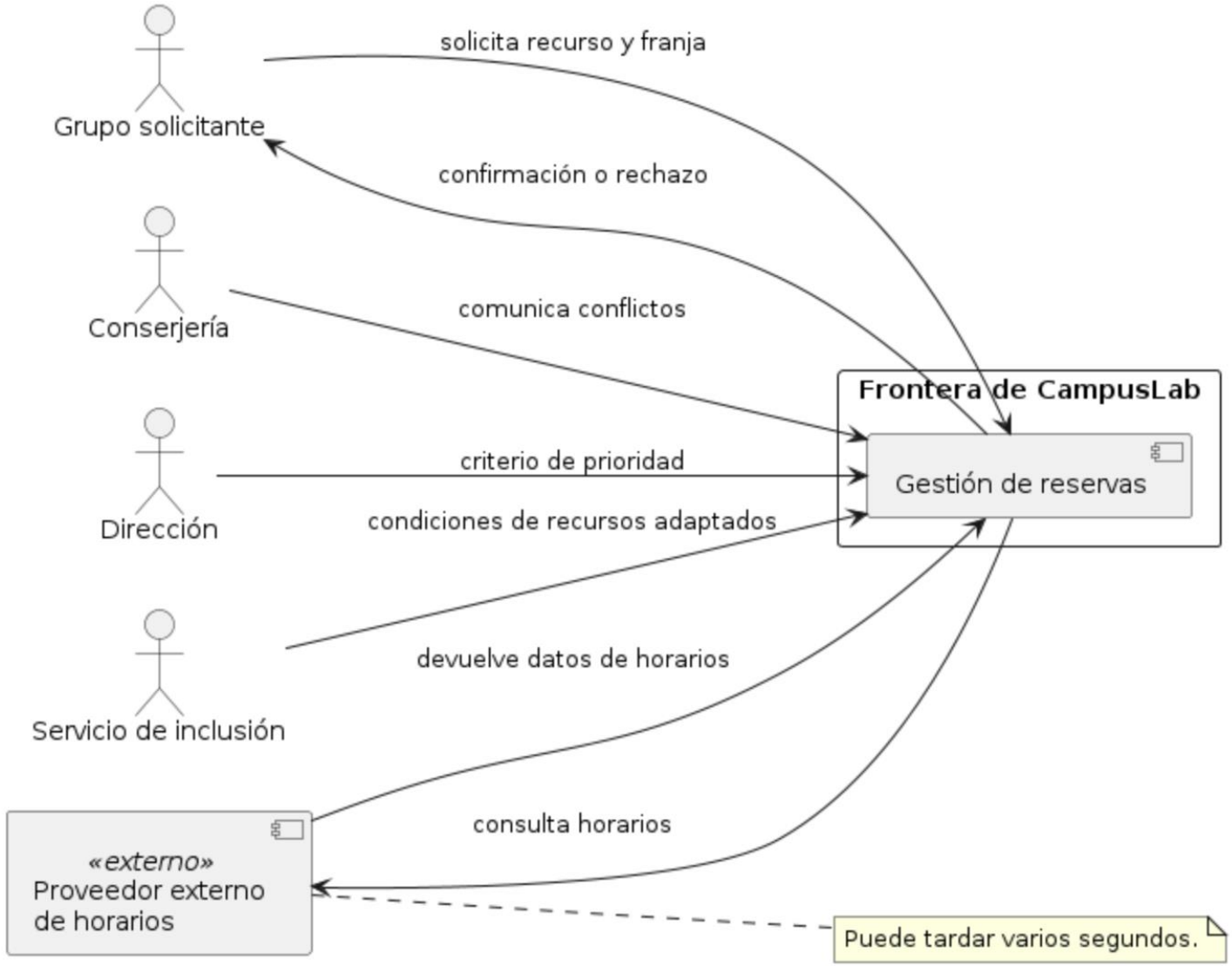
1.3 Modelos de representación

Pregunta	Modelo posible	Hallazgo buscado
¿Dónde acaba CampusLab?	Diagrama de contexto	Actores, sistemas externos y responsabilidades.
¿Cómo se realiza una tarea?	Flujo de actividad o recorrido	Pasos, decisiones, esperas y excepciones.
¿Qué significan los conceptos?	Modelo de dominio y glosario	Ambigüedades, relaciones y reglas.
¿Cómo cambia una reserva?	Máquina de estados	Transiciones imposibles o eventos omitidos.
¿Qué valor se entrega primero?	Mapa de historias	Flujo completo y cortes incrementales.
¿Se entiende la interacción?	Boceto o prototipo	Errores de comprensión antes de construir.

Veamos algunos ejemplos...

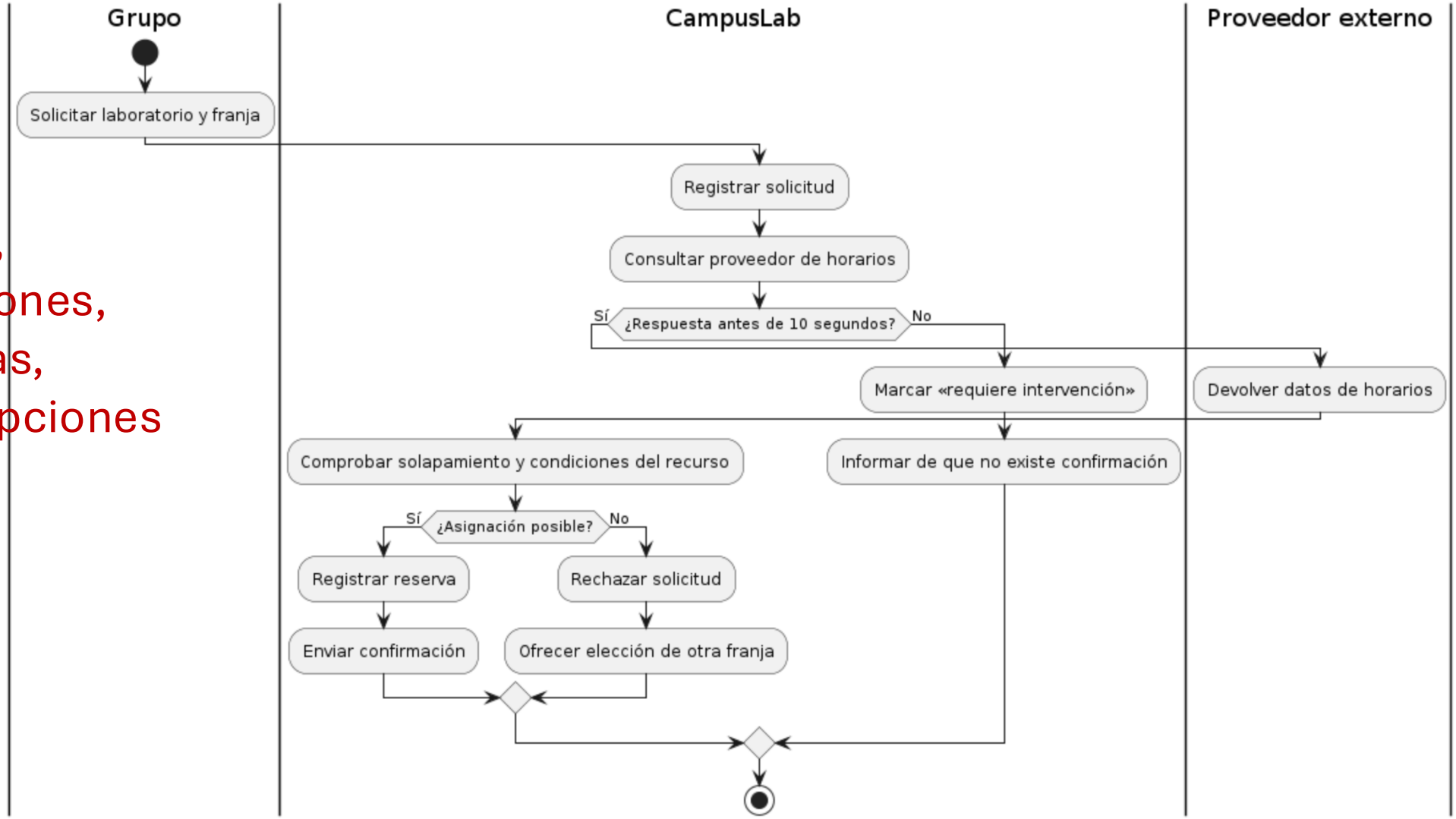
1.3 Modelos de representación:
diagrama de contexto

Actores,
sistemas externos
y responsabilidades



1.3 Modelos de representación:
flujo de actividad o recorrido

Pasos,
decisiones,
esperas,
y excepciones

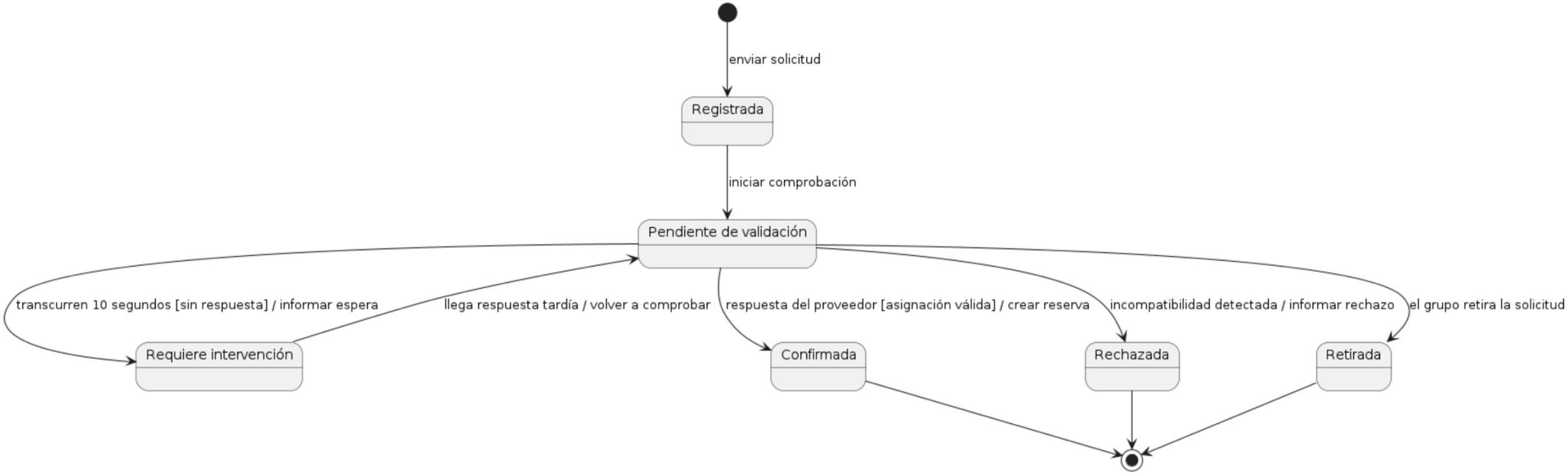


1.3 Modelos de representación:
modelo de dominio (diagrama de clases conceptual)

Ambigüedades,
relaciones
y reglas



1.3 Modelos de representación:
ciclo de vida (máquina de estados)



Reglas para modelos útiles

- Incluyen propósito, alcance, fecha y fuente.
- Usan el vocabulario del dominio definido en un glosario.
- Muestran incertidumbre; no presentan hipótesis como hechos.
- Mantienen solo el detalle necesario para la decisión.
- Se versionan junto con el cambio que explican.
- Se contrastan con ejemplos y contraejemplos.

Reglas para modelos útiles

- Incluyen propósito, alcance, fecha y fuente.
- Usan el vocabulario del dominio definido en un glosario.
- Muestran incertidumbre; no presentan hipótesis como hechos.
- Mantienen solo el detalle necesario para la decisión.
- Se versionan junto con el cambio que explican.
- Se contrastan con ejemplos y contraejemplos.

Dos modelos pueden ser **individualmente correctos** y **conjuntamente incompatibles**. Si la historia permite cancelar una reserva confirmada, pero la máquina de estados no contiene esa transición, existe una discrepancia que debe resolverse.

1.4 Requisitos de usuario y de software

Requisito de usuario: expresa, en un lenguaje comprensible para las personas interesadas, qué necesita lograr un perfil y qué condiciones hacen útil el resultado. Puede apoyarse en escenarios o modelos; no tiene por qué ser una frase vaga ni limitarse a una función.

1.4 Requisitos de usuario y de software

Requisito de usuario: expresa, en un lenguaje comprensible para las personas interesadas, qué necesita lograr un perfil y qué condiciones hacen útil el resultado. Puede apoyarse en escenarios o modelos; no tiene por qué ser una frase vaga ni limitarse a una función.

Requisito de software: concreta qué se debe proporcionar, con reglas, condiciones y criterios suficientes para que el equipo pueda implementarlo y verificarlo, sin imponer prematuramente una arquitectura.

> Si se habla de **requisitos del sistema** en sentido amplio, también pueden incluirse responsabilidades de personas, procedimientos y otros equipos; no todo corresponde al software.

1.4 Requisitos de usuario y de software

Nivel	Pregunta	Ejemplo en CampusLab	Evidencia
Negocio	¿Qué resultado necesita la organización?	Reducir cancelaciones causadas por conflictos de recursos.	Indicador operativo y objetivo.
Persona usuaria	¿Qué debe poder conseguir un perfil?	Un grupo sabe si dispone del laboratorio y puede buscar una alternativa si no.	Escenario o recorrido.
Software	¿Qué respuesta verificable debe proporcionar CampusLab?	Ante solicitudes incompatibles, confirma como máximo una y responde a las restantes.	Criterios y pruebas.
Componente	¿Qué responsabilidad técnica se asigna?	El servicio de reservas controla la exclusión.	Contrato de interfaz y prueba de integración.

Cuadro 1.6: Niveles de requisitos y evidencias asociadas.

1.4 Requisitos de usuario y de software

Nivel	Pregunta	Ejemplo en CampusLab	Evidencia
Negocio	¿Qué … organ…	…	…ativo
Persona usuaria	¿Qué … perfil…	…	…corri-
Software	¿Qué … debe … CampusLab?	… responde a las restantes.	…ebas.
Componente	¿Qué responsabilidad técnica se asigna?	El servicio de reservas controla la exclusión.	Contrato de interfaz y prueba de integración.

Cuadro 1.6: Niveles de requisitos y evidencias asociadas.

La relación no tiene por qué ser uno a uno

Un objetivo puede originar varios requisitos de usuario; uno de estos puede concretarse en varios requisitos de software; y un requisito de software puede distribuirse entre componentes.

1.4 Requisitos de usuario y de software; funcionales y no funcionales

El nivel no determina la naturaleza del requisito.

La misma necesidad puede expresarse en las cuatro combinaciones siguientes:

Nivel	Funcional: qué se logra o hace	No funcional: bajo qué condiciones resulta aceptable
Usuario	El grupo conoce si su solicitud ha sido confirmada y puede elegir otra franja tras un rechazo.	El grupo recibe una respuesta comprensible y a tiempo para reorganizar su actividad.
Software	CampusLab confirma como máximo una solicitud incompatible y comunica el resultado de las demás.	La latencia de la respuesta cumple el percentil y umbral acordados bajo la carga pico definida.

1.5 Requisitos funcionales y no funcionales

Requisito funcional.- Indica qué debe hacer el sistema: acciones, servicios, cálculos, reglas y respuestas observables ante entradas, eventos o situaciones concretas, incluidas las excepciones.

CampusLab: impedir dos confirmaciones incompatibles; comunicar el rechazo.

1.5 Requisitos funcionales y no funcionales

Requisito funcional.- Indica qué debe hacer el sistema: acciones, servicios, cálculos, reglas y respuestas observables ante entradas, eventos o situaciones concretas, incluidas las excepciones.

CampusLab: impedir dos confirmaciones incompatibles; comunicar el rechazo.

Requisito no funcional (o de calidad).- Indica bajo qué condiciones y con qué nivel de calidad deben resultar aceptables esas funciones: rendimiento, seguridad, usabilidad, fiabilidad, mantenibilidad, compatibilidad, etc.

CampusLab: que la respuesta tarde menos de X milisegundos.

1.5 Requisitos funcionales y no funcionales

Requisito funcional.- Indica qué debe hacer el sistema: acciones, servicios, cálculos, reglas y respuestas observables ante entradas, eventos o situaciones concretas, incluidas las excepciones.

CampusLab: impedir dos confirmaciones incompatibles; comunicar el rechazo.

Requisito no funcional (o de calidad).- Indica bajo qué condiciones y con qué nivel de calidad deben resultar aceptables esas funciones: rendimiento, seguridad, usabilidad, fiabilidad, mantenibilidad, compatibilidad, etc.

CampusLab: que la respuesta tarde menos de X.

Restricción- Limita el espacio de soluciones o las condiciones de operación por una obligación legal, contractual, tecnológica, organizativa o temporal.

Podría agruparse bajo "no funcional", pero conviene identificarla por separado: imponer un proveedor institucional no mide la calidad del servicio.

1.5 Requisitos funcionales y no funcionales

Conviene mantener esta distinción para no confundir implicaciones de la información:

Categoría	**Qué expresa**	**Ejemplo**
Requisito funcional	Comportamiento, cálculo o regla observable.	Impedir la cancelación cuando faltan menos de 24 horas.
Requisito de calidad	Grado de desempeño que debe alcanzarse en un escenario.	Confirmar el 95 % de las reservas en menos de un segundo bajo la carga acordada.
Restricción de solución	Límite que reduce las alternativas de diseño o implantación.	Autenticar mediante el proveedor institucional de identidad.
Obligación de proceso o evidencia	Condición sobre cómo se construye, revisa o demuestra el cumplimiento.	Conservar una revisión independiente y el resultado de las pruebas de autorización.

1.5 Requisitos funcionales y no funcionales

No todos los requisitos no funcionales se aplican por igual a todo el sistema. Debe indicarse si afectan a una operación, un conjunto de datos, un perfil, una integración, un entorno o el servicio completo.

1.5 Requisitos funcionales y no funcionales

No todos los requisitos no funcionales se aplican por igual a todo el sistema. Debe indicarse si afectan a una operación, un conjunto de datos, un perfil, una integración, un entorno o el servicio completo.

También **deben declararse las condiciones en las que se exigen** (carga normal o pico, horario de mantenimiento, pérdida de una dependencia, dispositivo asistivo, red degradada o recuperación tras un fallo).

Ej.: "*el sistema estará disponible el 99,9\%*" deja preguntas abiertas:

> qué servicio se mide, durante qué ventana, desde qué punto, qué interrupciones se excluyen y qué ocurre con una respuesta técnicamente correcta pero demasiado lenta.

Delimitar el alcance evita tanto promesas imposibles como pruebas que demuestran algo distinto de lo que importa.

1.5 Requisitos funcionales y no funcionales: **criterio de ajuste**

Todo requisito necesita una **forma acordada de decidir si se satisface**. El criterio puede ser una prueba automatizada, una inspección, una medición, una demostración o un análisis.

Si solo podemos evaluarlo mediante una impresión subjetiva, debemos refinarlo.

1.5 Requisitos funcionales y no funcionales: **criterio de ajuste**

Frase inicial	Problema	Reformulación verificable
La aplicación será rápida.	No indica operación, carga ni umbral.	Con 300 sesiones activas, el 95 % de las consultas de disponibilidad responde en menos de 800 ms.
El sistema será intuitivo.	"Intuitivo" depende del perfil y la tarea.	Ocho de diez personas de conserjería completan una reasignación en menos de 90 s, sin ayuda, tras una sesión de formación.
Los datos estarán seguros.	No identifica amenaza ni protección.	Un usuario sin rol administrativo no puede consultar el motivo privado de una reserva; todo intento queda auditado.
Nunca se perderán reservas.	Absoluto difícil de garantizar y medir.	Ante caída del proceso tras confirmar, la reserva aceptada aparece al recuperar el servicio y no se duplica el cargo.

Escenarios de atributos de calidad

Convierten una **preocupación amplia**, como rendimiento, disponibilidad, seguridad o modificabilidad, en una **situación concreta** que puede orientar arquitectura, pruebas y operación.

No es una categoría adicional de requisito ni una prueba aislada: es una **estructura para expresar bajo qué condiciones esperamos qué respuesta y cómo juzgaremos si resulta aceptable**.

Escenarios de atributos de calidad

Convierten una **preocupación amplia**, como rendimiento, disponibilidad, seguridad o modificabilidad, en una **situación concreta** que puede orientar arquitectura, pruebas y operación.

No es una categoría adicional de requisito ni una prueba aislada: es una **estructura para expresar bajo qué condiciones esperamos qué respuesta y cómo juzgaremos si resulta aceptable**.

Cuando [**fuente**] produce [**estímulo**]
sobre [**elemento afectado**] en [**entorno**],
el sistema [**respuesta**],
evaluada mediante [**medida de la respuesta**].

Escenarios de atributos de calidad

Elemento	Pregunta que responde	Ejemplo en CampusLab
Fuente	¿Quién o qué origina el evento?	Trescientas sesiones autenticadas de alumnado y personal.
Estímulo	¿Qué evento debe gestionar el sistema?	Consultas simultáneas de disponibilidad para una misma franja.
Entorno	¿En qué estado o condición sucede?	Periodo de matrícula, carga pico y proveedor de horarios operativo.
Elemento afectado	¿Qué parte del sistema recibe el estímulo?	Servicio de consulta y su caché de disponibilidad.
Respuesta	¿Qué comportamiento observable debe producirse?	Devuelve datos con antigüedad conocida, sin mezclar permisos ni bloquear confirmaciones.
Medida de respuesta	¿Qué magnitud y umbral permiten decidir?	Percentil 95 inferior a 800 ms, error menor del 0,5 % y antigüedad máxima de 30 s.

Escenarios de atributos de calidad

Elemento	Pregunta que responde	Ejemplo en CampusLab
Fuente	¿Quién o qué origina el evento?	Trescientas sesiones autenticadas de
Estímulo		dad
Entorno		
Elemento afectado		
Respuesta		cida,
Medida de respuesta	¿Qué magnitud y umbral permiten decidir?	Percentil 95 inferior a 800 ms, error menor del 0,5 % y antigüedad máxima de 30 s.

Convertir la calidad en escenarios obliga a **concretar condiciones y umbrales**.

Esa misma lógica conduce a una pregunta cuantitativa más amplia: cuánto trabajo, concurrencia y volumen debe soportar el sistema y qué debe ocurrir cuando alcance sus límites.

1.6 **Requisitos de capacidad** y restricciones de uso

La **capacidad** responde a una pregunta específica: **cuánto trabajo debe admitir el sistema, durante cuánto tiempo y qué comportamiento conservará al aproximarse a sus límites**. Puede referirse a:

- sesiones concurrentes;
- solicitudes por segundo y distribución de ráfagas;
- volumen de datos y crecimiento;
- tamaño máximo de una operación;
- número de integraciones o recursos gestionados;
- tiempo durante el que debe conservarse información;
- disponibilidad necesaria en periodos críticos.

1.6 **Requisitos de capacidad** y restricciones de uso

La **capacidad** responde a una pregunta específica: **cuánto trabajo debe admitir el sistema, durante cuánto tiempo y qué comportamiento conservará al aproximarse a sus límites**.

Una cifra aislada no permite diseñar ni probar capacidad: un modelo mínimo de carga combina perfiles, operaciones, frecuencia, concurrencia, distribución temporal y volumen de datos.

> Las medias ocultan picos: si casi todas las reservas ocurren diez minutos antes de una clase, la media diaria aporta poca información de diseño.

1.6 **Requisitos de capacidad** y restricciones de uso

La **capacidad** responde a una pregunta específica: **cuánto trabajo debe admitir el sistema, durante cuánto tiempo y qué comportamiento conservará al aproximarse a sus límites**.

Operación	**Carga normal**	**Pico previsto**	**Dato pendiente**
Consultar disponibilidad	4 solicitudes/s	25 solicitudes/s	Duración real del pico.
Confirmar reserva	0,5 solicitudes/s	5 solicitudes/s	Porcentaje de concurrencia sobre un recurso.
Importar horarios	Una vez al día	12.000 filas	Variabilidad del proveedor.
Generar informe	20 al día	10 simultáneos	Periodo máximo consultado.

Cuadro 1.11: Perfil inicial de capacidad con incertidumbre explícita.

1.6 Requisitos de capacidad y **restricciones de uso**

Restricción de uso: describe **condiciones en las que el producto debe funcionar** (dispositivos, conectividad, accesibilidad, idioma, entorno físico, formación disponible o interrupciones tolerables).

Una restricción de solución impone una elección: plataforma institucional, protocolo, ubicación de datos o tecnología aprobada.

Ej. "*Debe funcionar desde los navegadores instalados en las aulas*'" es una restricción de uso; "*debe implementarse con el framework X*'" restringe la solución.

1.6 Requisitos de capacidad y **restricciones de uso**

Toda restricción debería incluir:

- fuente y autoridad;
- motivo;
- alcance y fecha de vigencia;
- método de verificación;
- persona que puede aprobar una excepción

¿Calidad? De la carga nominal a la saturación

La capacidad útil no es el máximo obtenido en una prueba aislada.

Hay que distinguir:

- **Carga nominal**: nivel habitual para el que se espera margen operativo.
- **Pico previsto**: ráfaga conocida que debe absorberse durante un periodo definido.
- Punto de saturación: nivel a partir del cual colas, errores o consumo crecen de forma no aceptable.
- **Modo degradado**: funciones que se conservan, limitan o desactivan para proteger integridad y recuperación.
- **Recuperación**: tiempo y procedimiento para volver a un estado estable después del pico.

¿Calidad? De la carga nominal a la saturación

La capacidad útil no es el máximo obtenido en una prueba aislada.

Un **requisito de capacidad** está suficientemente definido cuando identifica operaciones y perfiles, distribución temporal, datos, entorno, umbrales, modo degradado, recuperación y método de medida.

Las cifras sin fuente permanecen como hipótesis y deben incluir una estrategia para revisarlas.

1.7 Documentación y revisión de requisitos

Objetivo de documentar: conservar una comprensión compartida suficiente para decidir, construir, verificar y mantener.

Documentar, revisar y trazar son actividades transversales: comienzan al registrar el primer hecho y continúan cuando una incidencia de producción cuestiona lo acordado.

Para documentar:

1) Comprobar si cada requisito puede interpretarse y verificarse.
2) Revisar si el conjunto conserva coherencia, trazabilidad y suficiente cobertura para sostener una decisión.

1.7 Documentación y revisión de requisitos

Para lograr esto, **un requisito individual debe ser**:

- **necesario**: responde a un objetivo o riesgo identificable;
- **claro**: los perfiles relevantes lo interpretan de forma compatible;
- **singular**: expresa una obligación principal;
- **factible**: puede satisfacerse dentro de restricciones conocidas;
- **verificable**: existe un método realista para evaluarlo;
- **trazable**: conocemos su origen, relaciones y evolución;
- **modificable**: puede cambiarse sin inconsistencias ocultas

1.7 Documentación y revisión de requisitos

Para lograr esto, **un requisito individual debe ser**:

- **necesario**: responde a un objetivo o riesgo identificable;
- **claro**: los perfiles relevantes lo interpretan de forma compatible;
- **singular**: expresa una obligación principal;
- **factible**: puede satisfacerse dentro de restricciones conocidas;
- **verificable**: existe un método realista para evaluarlo;
- **trazable**: conocemos su origen, relaciones y evolución;
- **modificable**: puede cambiarse sin inconsistencias ocultas

Además, el conjunto debe ser suficientemente completo, consistente, organizado y comprensible.

1.7 Documentación y revisión de requisitos

Plantilla de lenguaje natural para definir requisitos:

Cuando [condición o evento], el sistema [respuesta observable] para [actor u objetivo], respetando [restricciones]. Se considerará satisfecho si [criterio y método de verificación].

Ejemplo con CampusLab:

Cuando dos solicitudes intenten confirmar el mismo recurso y franja, CampusLab aceptará como máximo una reserva. La solicitud rechazada conservará sus datos y recibirá una alternativa o un motivo accionable. Se verificará mediante una prueba de integración concurrente repetida cien veces sin dobles confirmaciones.

1.7 Documentación y revisión de requisitos

Plantilla de lenguaje natural para definir requisitos:

Cuando [condición o evento], el sistema [respuesta observable] para [actor u objetivo], respetando [restricciones]. Se considerará satisfecho si [criterio y método de verificación].

En estos casos, cuidado con:

- Palabras vagas (ambigüedades)
- Términos absolutos (siempre, nunca…)
- Pasivas sin responsable (“Se informará…”)
- Soluciones disfrazadas (impone cosas)
- Requisitos múltiples
- Excepciones ocultas

1.7 Documentación y revisión de requisitos

Revisión de requisitos: buscar defectos que pueden causar decisiones incorrectas. Proceso:

1. Preparar: definir objetivo, alcance y perfiles revisores.
2. Recorrer: usar ejemplos concretos, datos límite y escenarios alternativos.
3. Desafiar: preguntar qué falta, qué contradice y cómo se verificará.
4. Clasificar: defecto, pregunta, mejora o decisión.
5. Resolver: asignar responsable y fecha, sin esconder asuntos abiertos.
6. Confirmar: comprobar el conjunto tras los cambios.

1.7 Documentación y revisión de requisitos

Revisión de requisitos: buscar defectos que pueden causar decisiones incorrectas. Proceso:

Es importante que los requisitos los revisen perfiles variados, ya que cada uno encuentra defectos distintos:

- Usuario: regla omitida
- Desarrollador: dependencias omitidas
- Pruebas: criterio no observable
- Operaciones: condición no monitorizable
- Seguridad: exposición indebida de datos

1.7 Documentación y revisión de requisitos

Trazabilidad: relaciona objetivos, requisitos, decisiones, elementos de diseño, código, pruebas, incidencias y entregas.

- Ideal como herramienta para trazabilidad: repositorio

1.8 Indicadores clave (KPI) y resultados

Indicador clave de rendimiento (KPI, Key Performance Indicator): es un indicador seleccionado porque informa sobre el avance de un objetivo relevante y puede desencadenar una decisión.

No toda métrica es un KPI: contar solicitudes, commits o tiempo de respuesta produce datos útiles, pero solo se convierte en indicador clave cuando existe una relación razonada con un resultado y alguien actuará según su valor.

1.8 Indicadores clave (KPI) y resultados

Un KPI bien definido necesita, como mínimo:

- objetivo al que sirve y decisión que pretende orientar;
- población, fórmula, unidad y fuente de datos;
- ventana temporal y frecuencia de revisión;
- línea base de medida (valor inicial), objetivo o intervalo esperado;
- responsable de interpretarlo y acción ante una desviación;
- limitaciones, posibles sesgos y métricas de equilibrio.

1.8 Indicadores clave (KPI) y resultados

Para CampusLab, "*reducir cancelaciones causadas por conflictos de recursos*" puede observarse mediante el porcentaje de reservas confirmadas que terminan canceladas por una colisión atribuible al sistema:

KPIcolisiones = Ccolision / Rconfirmadas * 100

donde

Ccolision: número de reservas canceladas por una colisión atribuible al sistema

Rconfirmadas: total de reservas confirmadas en la misma ventana

1.8 Indicadores clave (KPI) y resultados

Para CampusLab, "*reducir cancelaciones causadas por conflictos de recursos*" puede observarse mediante el porcentaje de reservas confirmadas que terminan canceladas por una colisión atribuible al sistema:

KPIcolisiones = Ccolision / Rconfirmadas * 100

Un KPI útil orienta una decisión
sin permitir que el equipo optimice la cifra
destruyendo el resultado que pretendía protege

1.9 Retomando las perspectivas de trabajo

Decisión	Predictiva	Adaptativa	Híbrida
Captación	Campaña amplia antes de la línea base; nuevos hallazgos pasan por control de cambios.	Descubrimiento continuo; se reduce primero la incertidumbre del próximo incremento.	Obligaciones y dependencias se investigan pronto; la experiencia se explora por ciclos.
Modelos y requisitos	Modelos revisados y requisitos detallados como conjunto, con cobertura y consistencia antes del compromiso.	Modelos y detalles se actualizan cuando sostienen la próxima decisión; no se aplaza la calidad crítica.	Contexto, contratos e invariantes estables; recorridos y ejemplos evolucionan.
Capacidad y calidad	Perfil estimado para dimensionar y contratar; cambios pueden afectar a la línea base.	Límites críticos iniciales, instrumentación y revisión de umbrales con datos de incrementos.	Compromisos contractuales fijos y estrategias internas ajustables.
Documentar y trazar	Especificación versionada; cada cambio registra impacto, autoridad y pruebas afectadas.	Visión, backlog, modelos, decisiones y pruebas mantienen navegable el estado actual.	Control formal de obligaciones; evolución ligera de detalles reversibles.

1.9 Algunos conceptos importantes

Stakeholder o parte interesada: cualquier persona u organización con derechos, responsabilidades, interés o capacidad de verse afectada por el sistema. No equivale únicamente a “cliente” ni a “usuario final”, y aportar información no implica autoridad para aprobar cualquier requisito.

Tipos de perfiles stakeholder:

- quienes usan o reciben directamente el resultado;
- quienes operan, despliegan, mantienen o dan soporte;
- quienes pagan, dirigen, auditan o regulan;
- sistemas externos, proveedores y propietarios de datos;
- personas que pueden quedar excluidas, perjudicadas o expuestas a un riesgo.

1.9 Algunos conceptos importantes

SLA (Service Level Agreement) o acuerdo de nivel de servicio: define el servicio esperado, niveles medibles, responsabilidades y mecanismos de gestión cuando no se cumplen.

Para interpretarlo y auditarlo debe especificar:

- servicio, operaciones, perfiles, datos y entornos incluidos;
- punto de observación, fuente de datos y método de cálculo;
- objetivo, ventana temporal, agregación y tolerancias;
- exclusiones, mantenimiento planificado y tratamiento de dependencias;
- responsabilidades, comunicación, escalado y tiempos de respuesta;
- consecuencias o acciones correctivas ante incumplimiento;
- vigncia y procedimiento de revisión.

1.9 Algunos conceptos importantes

- **Objetivo de calidad**: umbral observable exigido por un escenario, *como responder en menos de 800 ms bajo una carga definida*.
- **Presupuesto técnico**: reparto de un límite cuantitativo entre componentes, etapas o causas. Se nombra por su magnitud: presupuesto de latencia, capacidad, indisponibilidad, memoria o errores; “presupuesto de calidad” resulta demasiado genérico.
- **Compromiso entre atributos:** decisión en la que mejorar un atributo empeora otro, consume más recursos o reduce opciones futuras. No siempre existe oposición inevitable.
- **Coste del compromiso:** consecuencia de la alternativa elegida (dinero, tiempo, complejidad, exposición, pérdida de precisión, carga operativa o riesgo residual).

1.10 Historias de usuario (perspectivas ágiles)

Es una unidad de trabajo y aprendizaje que expresa una capacidad valiosa desde la perspectiva de un perfil, pero no determina por sí sola todo el comportamiento del software.

Se identifica mediante una tarjeta, se desarrolla mediante conversación y se confirma con ejemplos o pruebas.

No es una especificación completa, una plantilla universal ni una unidad obligatoria de programación.

Presuponen colaboración frecuente con personas capaces de aclarar la necesidad, planificación evolutiva, entregas incrementales y retroalimentación rápida.

1.10 Historias de usuario (perspectivas ágiles)

Enfoque	Tratamiento principal	Papel de la historia de usuario
Predictivo o dirigido por plan	Se desarrolla una especificación estructurada, se revisa como conjunto, se establece una línea base y los cambios se someten a análisis de impacto y aprobación.	Puede utilizarse como ejemplo, resumen o ayuda para explorar necesidades, pero no sustituye automáticamente la especificación ni su trazabilidad.
Adaptativo o ágil	El conocimiento evoluciona en un backlog. Se detalla cerca de la implementación mediante conversación, ejemplos y retroalimentación de incrementos anteriores.	Es un mecanismo central para descubrir, negociar, dividir, planificar y confirmar una capacidad valiosa. La información puede distribuirse entre tarjeta, conversación y artefactos enlazados.
Híbrido	Se fijan pronto obligaciones estables, interfaces, invariantes y requisitos de calidad críticos; los detalles reversibles evolucionan por incrementos.	Organiza incrementos de valor, pero cada historia debe respetar y enlazar las restricciones y decisiones mantenidas fuera de ella.

1.10 Historias de usuario (perspectivas ágiles)

LAS TRES C de las historias de usuario:

- **Card** (tarjeta). Un identificador y una frase breve que permiten localizar, ordenar y recordar la capacidad. No pretende almacenar todo el conocimiento.
- **Conversation** (conversación). Preguntas, ejemplos, reglas, bocetos, modelos, alternativas y acuerdos construidos durante el tiempo necesario. Puede dejar evidencia escrita y evolucionar al aparecer nueva información.
- **Confirmation** (confirmación). Criterios, ejemplos y pruebas que permiten juzgar si la capacidad y sus condiciones relevantes están satisfechas.

1.10 Historias de usuario (perspectivas ágiles)

INVEST de las historias de usuario:

- Independent (independiente): la historia debe ser autónoma. No debe depender de otras tareas para poder desarrollarse, lo que permite planificarla e implementarla en cualquier orden.

1.10 Historias de usuario (perspectivas ágiles)

INVEST de las historias de usuario:

- Independent (independiente): la historia debe ser autónoma. No debe depender de otras tareas para poder desarrollarse, lo que permite planificarla e implementarla en cualquier orden.
- Negotiable (negociable): no es un contrato cerrado de requerimientos. Es una base abierta a la conversación y el acuerdo continuo.

1.10 Historias de usuario (perspectivas ágiles)

INVEST de las historias de usuario:

- Independent (independiente): la historia debe ser autónoma. No debe depender de otras tareas para poder desarrollarse, lo que permite planificarla e implementarla en cualquier orden.
- Negotiable (negociable): no es un contrato cerrado de requerimientos. Es una base abierta a la conversación y el acuerdo continuo.
- Valuable (valiosa): debe aportar un beneficio claro y real para el usuario final o el cliente del producto.

1.10 Historias de usuario (perspectivas ágiles)

INVEST de las historias de usuario:

- Independent (independiente): la historia debe ser autónoma. No debe depender de otras tareas para poder desarrollarse, lo que permite planificarla e implementarla en cualquier orden.
- Negotiable (negociable): no es un contrato cerrado de requerimientos. Es una base abierta a la conversación y el acuerdo continuo.
- Valuable (valiosa): debe aportar un beneficio claro y real para el usuario final o el cliente del producto.
- Estimable: el equipo técnico debe tener la información suficiente para calcular el esfuerzo necesario para completarla.

1.10 Historias de usuario (perspectivas ágiles)

INVEST de las historias de usuario:

- Independent (independiente): la historia debe ser autónoma. No debe depender de otras tareas para poder desarrollarse, lo que permite planificarla e implementarla en cualquier orden.
- Negotiable (negociable): no es un contrato cerrado de requerimientos. Es una base abierta a la conversación y el acuerdo continuo.
- Valuable (valiosa): debe aportar un beneficio claro y real para el usuario final o el cliente del producto.
- Estimable: el equipo técnico debe tener la información suficiente para calcular el esfuerzo necesario para completarla.
- Small (pequeña): debe tener el tamaño adecuado para realizarse en poco tiempo (idealmente en pocos días o dentro de un solo sprint). Las historias muy grandes se consideran épicas y deben dividirse.

1.10 Historias de usuario (perspectivas ágiles)

INVEST de las historias de usuario:

- Independent (independiente): la historia debe ser autónoma. No debe depender de otras tareas para poder desarrollarse, lo que permite planificarla e implementarla en cualquier orden.
- Negotiable (negociable): no es un contrato cerrado de requerimientos. Es una base abierta a la conversación y el acuerdo continuo.
- Valuable (valiosa): debe aportar un beneficio claro y real para el usuario final o el cliente del producto.
- Estimable: el equipo técnico debe tener la información suficiente para calcular el esfuerzo necesario para completarla.
- Small (pequeña): debe tener el tamaño adecuado para realizarse en poco tiempo (idealmente en pocos días o dentro de un solo sprint). Las historias muy grandes se consideran épicas y deben dividirse.
- Testable (comprobable): debe incluir criterios de aceptación claros que permitan verificar mediante pruebas si funciona y cumple con lo esperado.

1.11 Product backlog y priorización (perspectivas ágiles)

Product backlog: es una representación ordenada y evolutiva del trabajo necesario para mejorar un producto.

Puede incluir capacidades, defectos, riesgos, experimentos, deuda técnica, aprendizaje y trabajo operativo.

1.11 Product backlog y priorización (perspectivas ágiles)

¿Cómo se **prioriza** cada requisito, tarea o funcionalidad en el product backlog?

MoSCoW (Must, Should, Could, Won't):

- **Mo - Must have (Imprescindible):** requisitos críticos y no negociables. Sin ellos, el producto no funciona, no es viable o carece de sentido (clave para el Producto Mínimo Viable o MVP).
- **S - Should have (Deseable / Importante):** características importantes y de gran valor, pero no vitales para el lanzamiento inmediato. Pueden posponerse si el tiempo es corto.
- **Co - Could have (Opcional / Si es posible):** funcionalidades secundarias o "nice-to-have". Se incluyen solo si hay tiempo y recursos sobrantes.
- **W - Won't have (No se hará ahora):** elementos descartados para la fase actual. Aportan poco valor o exceden el presupuesto, aunque pueden considerarse para el futuro.